

Source Code Minimaps at Scale: Feature-Oriented Analysis of 100 Projects

Munif Gebara Jr.¹ , Igor S. Wiese² , Hua-Liang Wei³ , and
Yandre M. G. Costa^{1,4} 

¹ Graduate Program in Computer Science,
State University of Maringá (UEM), Maringá, Brazil.

² Federal University of Technology (UTFPR), Campo Mourão, Brazil.

³ School of Electrical and Electronic Engineering and Centre of Machine Intelligence
(CMI), The University of Sheffield, Sheffield, UK.

⁴ Department of Informatics, State University of Maringá (UEM), Maringá, Brazil.
✉ pg55752@uem.br

Abstract. In this work, we present a large-scale study of *source code minimaps* that spans 685,906 images from 100 open-source projects across 10 programming languages. The corpus is standardized to 128×128 pixels, anonymized, and accompanied by a file-level catalog that enables three complementary objectives: *type* (63 classes), *project* (99), and *author* (97). We evaluate convolutional backbones under a unified protocol (batch size 64, maximum 100 epochs, early stopping with patience = 5), comparing from-scratch and ImageNet-pretrained variants. The best validation results are obtained with EfficientNet-B0 (pretrained), reaching accuracies of 0.9333 (*type*), 0.8604 (*project*), and 0.8705 (*author*); ROC-AUC (macro/micro) remains near or above 0.995 across tasks, and optimizer choice (Adam vs. SGD) has negligible impact at the adopted precision. To interpret the models, we compute Integrated Gradients and aggregate attributions over the validation set, revealing that predictions for *type* rely on structural bands (indentation, delimiters), *project* emphasizes repository conventions and boilerplate, and *author* exhibits more diffuse stylistic cues. These findings establish a reproducible benchmark for learning from visual abstractions of code under severe class imbalance and motivate future work on imbalance-aware objectives, dataset scale and class imbalance; we also provide post-hoc attributions with Integrated Gradients. Code and dataset are publicly available at GitHub and Zenodo.

Keywords: Transfer learning · Class imbalance · Integrated Gradients · Software visualization.

1 Introduction

Software systems are becoming increasingly complex, spanning diverse domains and relying on heterogeneous technologies. Understanding and analyzing these systems at scale requires novel approaches that go beyond traditional static

analysis or rule-based tools. Recent studies have shown the potential of source code minimaps, which transform code into compact visual abstractions that capture structural and syntactic patterns. These visual signatures enable the application of machine learning (ML) and computer vision techniques to source code, supporting tasks such as stereotype identification, architectural analysis, and anomaly detection. However, despite promising results, most existing efforts remain limited to small datasets or shallow models, offering little interpretability regarding the internal decision-making processes of classifiers. [3]

Software visualization has long been a key strategy to support program comprehension and software evolution. Novais et al. [10] proposed the SourceMiner Evolution tool, which employs coordinated visualizations such as Treemaps and Polymetric Views to reveal structural and evolutionary aspects of software systems. Similarly, other visualization approaches such as SArF Map [9] explored feature-oriented clustering and user feedback visualization, reinforcing the role of visual abstractions in enhancing the understanding of large-scale systems. While effective, these methods typically require predefined rules, manual curation, or domain-specific mappings, which limit their scalability and adaptability. Our work builds on this foundation by employing source code minimaps as generic visual signatures that can be automatically processed by machine learning models.

In parallel, several studies investigated the transformation of non-visual data into visual domains for machine learning applications. Costa et al. [4,5] demonstrated how spectrograms of audio signals, combined with texture descriptors such as Local Binary Patterns (LBP), can achieve competitive results in music genre classification. This methodology inspired the application of texture-based techniques to software minimaps, as introduced in our previous work [3]. However, prior approaches were limited to a relatively small dataset and classical ML models, providing little insight into model decision-making. More recent advances in Explainable Artificial Intelligence (XAI) [1,13] highlight the importance of interpretability, particularly when deep learning models are applied to high-dimensional data. In this context, our study expands on earlier contributions by introducing a large dataset of 100 projects in 10 languages, evaluating deep learning architectures, and applying XAI methods such as Integrated Gradients to interpret how visual code patterns influence classification outcomes.

These developments motivate a next step that unifies scale and interpretability. We therefore extend minimap-based learning with a substantially larger corpus and modern deep networks, and we integrate XAI to audit model behavior, clarifying *which* visual cues are leveraged and *why* predictions differ across objectives.

To address these limitations, we introduce a large-scale dataset of source code minimaps comprising 100 projects across 10 programming languages, including Java, Python, JavaScript, C, C#, C++, PHP, Go, Ruby, and Kotlin. This dataset enables comprehensive evaluation of machine learning methods under realistic and diverse conditions. We investigate state-of-the-art deep learning models trained directly on minimaps and employ Explainable Artificial Intelli-

gence (XAI) techniques, such as Integrated Gradients, to analyze which visual patterns contribute most to classification decisions. This combination of dataset expansion, deep learning, and interpretability provides new insights into how visual representations of source code can support feature detection and software comprehension at scale.

To provide an overview of the workflow adopted in this study, Figure 1 illustrates the main stages of our approach. The process begins with the selection of programming languages and repositories, followed by the generation of minimaps and associated metadata. From this, relevant attributes are chosen for classification tasks, leading to the training phase. Finally, post-hoc explainability (XAI) methods are applied to interpret the results.

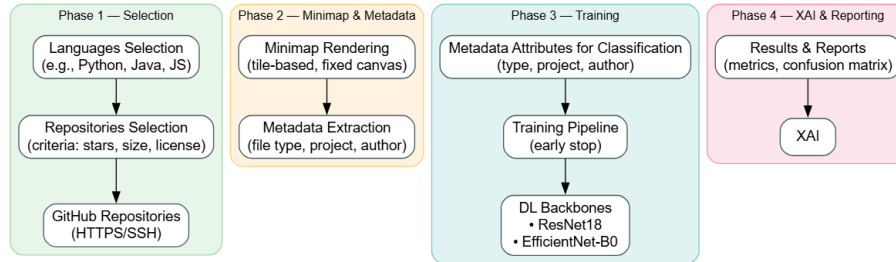


Fig. 1: Overview of the workflow adopted in this study.

The process begins with *repository selection*. A corpus of 100 public repositories across 10 programming languages is considered, prioritizing active maintenance, conventional tooling, and clear licensing. Language coverage is balanced as far as possible. Forks and mirrors are excluded to prevent duplication, and for each repository, an identifier and basic metadata (hosting platform, number of stars, default branch) are recorded.

Next comes *file filtering and annotation*. Source files are retained according to language-appropriate extensions, whereas generated, vendored, and build artifacts (e.g., `node_modules/`, `dist/`, `build/`, and VCS directories) are removed. Binary content is excluded via MIME inspection, and near-duplicates are eliminated using content hashes. Each remaining file receives three labels corresponding to the supervised objectives: a *type* label representing a coarse file role, a *project* label corresponding to the repository identifier, and an *author* label derived from version-control blame using last-committer attribution. File paths are stored as salted hashes to avoid leaking sensitive names.

Anonymization focuses on what is replaced versus what is preserved. Identifiers, string literals, and comments are substituted by placeholders, while whitespace, indentation, line counts, punctuation, and block delimiters are retained. The transformation is deterministic and language-aware so that structural and

layout cues remain available to the downstream vision models even though lexical content is suppressed.

For *minimap generation*, files are rendered as images with a fixed resolution of 128×128 pixels by default. **Grayscale** rendering is the standard setting, with RGB available as an option when explicitly required by an experiment. Short files are padded to the target canvas; long files are uniformly downsampled to preserve global layout. Pixel intensities are normalized to the $[0, 1]$ range. Variable resolutions are supported for ablation studies but are not used in the main benchmark.

The *catalog and dataset splits* record, for each image, the repository identifier, a file identifier (SHA-256), a hashed relative path, the programming language, and the labels for the three objectives (*type*, *project*, *author*). Splits are stratified per objective. To prevent leakage, the *project* task enforces repository disjointness across train, validation, and test, and the *author* task avoids placing the same author in both training and test partitions.

Training and evaluation follow a standardized protocol. Images are resized to 128×128 ; random horizontal flips are applied with probability 0.5, while vertical flips are disabled to respect layout semantics. The default backbone is **ResNet-18**; **EfficientNet-B0** and **ViT-B/16** are available as alternatives. Optimization uses Adam with learning rate 10^{-3} and weight decay 10^{-4} ; mini-batches contain 256 images and training runs for up to 100 epochs with early stopping (patience 10) on validation macro-F1. Class imbalance is handled with weighted cross-entropy using inverse-frequency weights, and imbalance is reported via IR_{global} and IR_{avg} . Three random seeds are executed and results are reported as mean $\pm$ standard deviation. The computational environment (GPU/CPU model, OS, and library versions) is described in a dedicated subsection for reproducibility.

Finally, *explainability* is integrated through Integrated Gradients configured with a black baseline and 50 steps, using the same input normalization as training. Attributions are rescaled to $[0, 1]$ and overlaid on the input image to highlight class-relevant regions. Parameters are kept constant across the three objectives to ensure qualitative comparability.

The remainder of this paper is organized as follows: Section 3 presents the dataset; Section 4 describes the training protocol and results; Section 5 introduces the explainability analyses; and Section 6 provides conclusions and directions for future work.

To facilitate reproducibility and reuse, we release both the source code and the full dataset. The training and evaluation pipeline, including configuration files and scripts for generating minimaps and reports, is available at

<https://anonymous.4open.science/r/minimap-learner-B626/README.md>.

The curated dataset (catalog and images) can be accessed and cited via Zenodo at <https://zenodo.org/records/16929672>. These resources enable end-to-end replication of our experiments and support further analyses.

The central contribution is a scalable, framework-agnostic methodology whose primary advantage is its reproducibility across **ten** distinct programming lan-

guages, indicating robustness beyond any single ecosystem. A corpus of 685906 minimaps drawn from 100 projects is assembled, together with a catalog and executable scripts that can be rerun on new repositories without language-specific tailoring, supporting three supervised objectives—*type*, *project*, and *author*. The anonymization stage removes lexical content while preserving structural and visual cues essential for learning; training and evaluation follow a fixed protocol with explicit imbalance reporting (IR_{global} , IR_{avg}); and Integrated Gradients is incorporated into the benchmark to expose class-relevant evidence in the images. Taken together, these choices render the approach robust and genuinely language-agnostic, facilitating faithful replication across heterogeneous codebases and toolchains.

Experiments were conducted on a workstation equipped with a single NVIDIA RTX 3070 (8 GB) GPU, a 6-core CPU, and 64 GB of RAM, running Ubuntu 24.04 LTS. The software stack comprised Python 3.11, PyTorch 2.x with CUDA 12.x, TorchVision, and Captum for explainability. Random seeds were fixed across runs, and early stopping criteria were identical for all model families to limit variance due to training dynamics.

2 Related Work

Software visualization has long supported program comprehension, especially for reasoning about feature evolution and maintenance. Novais et al. [11] demonstrated this with SourceMiner Evolution (SME), which offers multiple interactive and coordinated views to analyze how feature code changes across versions. These visual metaphors—including Treemaps, Polymetric views, and dependency graphs—highlight how structure and evolution become more tractable when exposed visually. In contrast to interactive, preconfigured views, the present approach leverages automated learning directly from source code minimaps rendered at fixed resolution, aiming to infer software stereotypes without manual mappings or hand-crafted dashboards. The result is a pipeline designed to be framework-agnostic and reproducible across languages and repositories.

Architectural specificities also shape how design problems surface in practice. Aniche et al. [2] curated a catalog of code smells for Model–View–Controller (MVC) architectures, showing that roles such as controllers and repositories follow distinct failure modes that generic smell catalogs may miss. While such domain knowledge remains valuable, the current approach diverges methodologically by using automated learning over minimaps to detect recurrent stereotypes from data itself, rather than relying on expert-defined catalogs. This data-driven route seeks scalability and adaptability to evolving practices, complementing architecture-specific insights reported in [2].

A parallel thread relevant to the methodology is the translation of non-visual phenomena into images that admit texture-based analysis and machine learning. Costa et al. [7] transformed audio signals into spectrograms for music genre classification, illustrating how time–frequency structure can be made amenable to visual pattern recognition. Subsequent work applied Local Binary Patterns (LBP)

with shallow classifiers such as SVMs to those spectrograms [6], underscoring the utility of texture descriptors in image-like encodings. The present contribution follows the same high-level premise — representing complex artifacts as images— but relies on end-to-end deep models (e.g., ResNet, EfficientNet, ViT) trained on fixed-size source code minimaps, coupled with Integrated Gradients to make class-relevant evidence explicit. This shift from hand-crafted features to learned representations enables scaling across heterogeneous codebases while retaining interpretability via XAI.

3 Dataset

The dataset used in this study was designed to provide large-scale and diverse resources for evaluating source code minimaps. It comprises a total of 685,906 minimap images extracted from 100 open-source projects, covering 63 distinct file types. In addition, contributions from 97 different authors were included, ensuring that the dataset reflects a variety of ecosystems, coding conventions, and development practices. This diversity enables the evaluation of machine learning models under realistic and heterogeneous conditions.

Projects and file types were selected to maximize representativeness across domains and programming paradigms. Figure 2 shows the Top-20 class frequencies for each objective. The most frequent types are `java` (88,249 samples), `ruby` (59,216), `c-header` (47,909), `c` (45,814), and `python` (42,581). Among projects, the Linux kernel dominates with 76,011 samples, followed by GitLabHQ (52,918), Elasticsearch (31,493), Moodle (24,484), and Kubernetes (23,435). In terms of authorship, Linus Torvalds accounts for 76,011 samples, with additional large contributions from automated accounts such as GitLab Bot (52,918) and Kubernetes Prow Robot (23,435). These numbers highlight the scale and breadth of the dataset, combining human and automated contributions across ecosystems.

The generation of minimaps followed the methodology established in our previous work [3], with extensions for scalability. Each project was scanned recursively; files were identified based on their extension and annotations; and anonymization was applied before transformation into minimaps. Two versions were produced: variable-size images (reflecting the original file layout) and fixed-size images of 128×128 pixels. The fixed-size setting ensures compatibility with convolutional neural networks and other deep learning architectures that require consistent input dimensions, enabling fair comparison between texture-based descriptors and modern deep learning methods.

To protect confidentiality, an anonymization process replaced identifiers (e.g., class names, variables, numeric constants) with canonical symbols while preserving structural tokens (e.g., brackets, operators, keywords). As shown in previous experiments [3], anonymization does not significantly affect classification accuracy while enabling datasets derived from industrial or proprietary projects to be safely shared, balancing privacy and fidelity.

In contrast to our previous study, this release *explicitly ships a file-level catalog* that accompanies the minimap images. The catalog CSV (Table 1) provides,

for each sample, the image identifier, the inferred file *type* (e.g., `java`, `ruby`, `c-header`), the *project* label (e.g., `linux`, `gitlabhq`), the original *file_path* and *file_name*, and the *author*. This structured index enables reproducible joins across objectives (*type*, *project*, *author*) and supports simple aggregations (e.g., language distributions per project). In the provided catalog snapshot, we observe 685,906 entries, with 63 file types, 100 projects, and 97 authors.

Table 1: Catalog fields shipped with the dataset (CSV).

Field	Type	Description	Example
<code>image</code>	string	Minimap image filename (relative to 00012345.png the images root); unique per sample.	
<code>type</code>	categorical	File type/stereotype inferred from extension and heuristics.	<code>java</code>
<code>project</code>	categorical	Repository/project identifier (normalized, lowercase).	<code>linux</code>
<code>file_path</code>	string	Relative path to the original source file inside the repository.	<code>kernel/sched/core.c</code>
<code>file_name</code>	string	Base filename of the source file.	<code>core.c</code>
<code>author</code>	categorical	VCS author name/handle associated with the sample.	<code>Linus Torvalds</code>

3.1 Dataset Description

As expected, the *author* task presents the largest number of classes and the highest imbalance, whereas *type* is comparatively more balanced and structurally distinct. The dataset is therefore suitable for studying imbalance-aware learning strategies, as well as for assessing model sensitivity across different classification objectives.

Cross-objective comparison. Table 2 consolidates cardinality and imbalance for direct comparison across objectives.

We characterize the corpus introduced in Section 3 using class-frequency summaries per objective and a compact cross-objective comparison. The release contains **685,906** fixed-size minimaps (128×128) annotated along three complementary objectives: *author* (97 classes), *type* (63), and *project* (99). Following best practice for imbalanced data, we report dispersion and two imbalance indicators: Global Imbalance Ratio (IR_{global}), defined in Equation 1, and Average Imbalance Ratio (IR_{avg}), defined in Equation 2. IR_{global} is about the extremes (worst case), and IR_{avg} is about the overall imbalance level [8][12].

$$IR_{global} = \frac{\max_i n_i}{\min_i n_i} \quad (1)$$

Where: n_i is the number of samples in class i . If IR_{global} is equal to 1, the dataset is perfectly balanced. Larger values indicate stronger imbalance.

$$IR_{avg} = \frac{1}{K} \sum_{i=1}^K \frac{n_{\max}}{n_i} \quad (2)$$

Where: K is the number of classes, $n_{max} = \max_i n_i$.

More sensitive to how minority classes compare against the majority. To avoid redundancy while preserving intuition about the head of each distribution, Figures 2a, 2c and 2b show the Top-20 class frequencies for each objective. For a concise side-by-side view of cardinality and imbalance, Table 2 summarizes the per-objective statistics used throughout the paper.

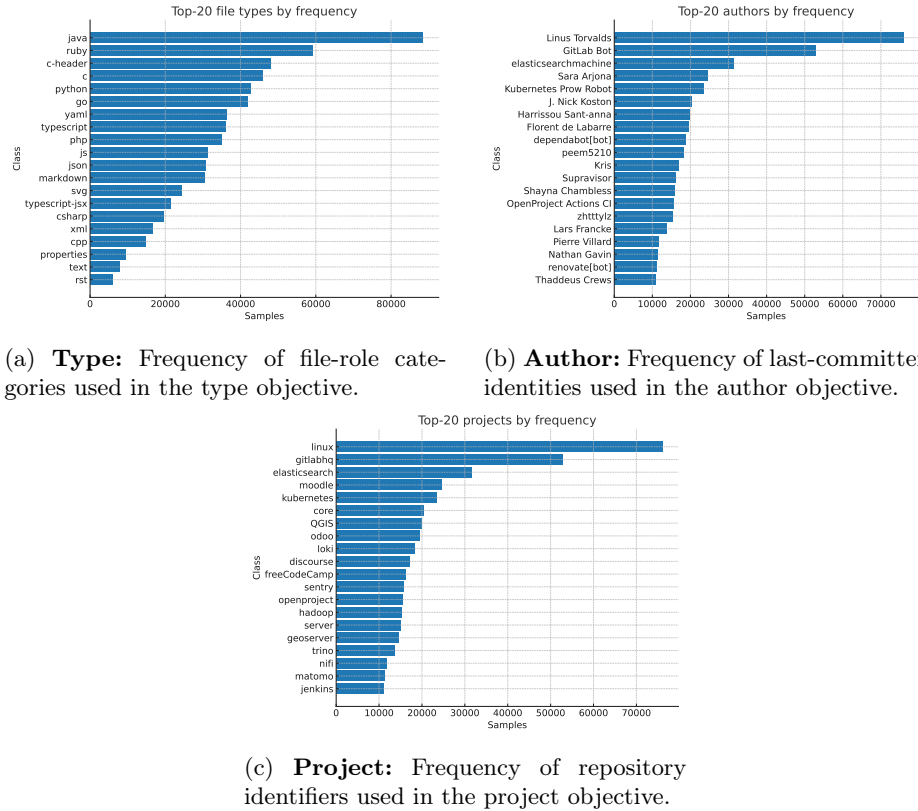


Fig. 2: Top-20 classes by frequency for the three objectives (*Type*, *Author*, *Project*). Head classes are highlighted to contextualize class imbalance across tasks.

4 Training Protocol and Results

We adopt a unified protocol across objectives and backbones to ensure comparability and statistical stability. Inputs are fixed-size minimaps (128×128 grayscale)

Table 2: Comparison across objectives (concise summary).

Objective	Classes	Samples	Median	Max	IR_{global}	IR_{avg}
Author	97	685,906	2,857	76,011	10,858.71	177.31
Type	63	685,906	1,120	88,249	88,249.00	4,402.12
Project	99	685,906	2,857	76,011	10,858.71	179.40

with input normalization to $[0, 1]$; models are fine-tuned end-to-end with task-specific linear heads; mixed-precision training (`torch.amp`) reduces memory footprint. Optimization uses Adam (10^{-3}) or SGD (momentum 0.9, 10^{-2}) with weight decay 10^{-4} . Training proceeds for at most 100 epochs with early stopping (patience = 10) based on validation accuracy; the best checkpoint is retained. We evaluate ResNet-18 (from-scratch and ImageNet-pretrained) and EfficientNet-B0 (ImageNet-pretrained) on three objectives: *type* (63 classes), *project* (99), and *author* (97); reporting Accuracy, ROC-AUC (macro/micro), and discussing macro-F1 and weighted-F1 as complementary measures under imbalance.

Metrics under imbalance. While accuracy and ROC-AUC provide a broad view of separability, class imbalance motivates macro-averaged measures. Macro-F1 equally weighs all classes, penalizing errors on minority classes; weighted-F1 accounts for support and tracks practical performance when head classes dominate. In our setting, macro-F1 typically lags behind micro metrics when distributions are long-tailed, especially for *project* and *author*, whereas *type* benefits from clearer structural cues (Section 3).

Consolidated training artifacts. The training overview figures summarize per-task validation metrics, learning curves (loss/accuracy), confusion matrices, and ROC curves for representative runs.

Quantitative summary. Table 3 consolidates validation metrics for two representative runs. In *type*, accuracies are ≥ 0.9236 with micro ROC-AUC ≥ 0.9988 , indicating strong separability of file-level structures. In *project*, accuracies lie in $[0.8513, 0.8557]$ with micro ROC-AUC ≈ 0.997 , reflecting heterogeneous repository conventions. In *author*, accuracies lie in $[0.8619, 0.8646]$, consistent with stylistic variability even under anonymization. At the adopted precision, optimizer differences are negligible; ImageNet pretraining yields consistent but modest gains in *type* and *project*.

Between-run differences. To make the comparison explicit, Table 4 reports the between-run deltas (run 20250817-012528 minus run 20250817-053841). We observe small but systematic improvements for *project*, and a slight trade-off in *author*; *type* remains consistently high with negligible AUC variation.

Discussion. Convergence profiles indicate that fixed-size minimaps and end-to-end fine-tuning are sufficient to capture discriminative, multi-scale patterns. The

Table 3: Validation metrics per objective for two representative runs.

Run	Task	Acc.	ROC-AUC (macro/micro)
run-20250817-053841	type	0.9236	0.9974 / 0.9989
run-20250817-053841	proj_orig	0.8513	0.9950 / 0.9964
run-20250817-053841	author	0.8646	0.9947 / 0.9963
run-20250817-012528	type	0.9263	0.9975 / 0.9988
run-20250817-012528	proj_orig	0.8557	0.9954 / 0.9968
run-20250817-012528	author	0.8619	0.9947 / 0.9963

Table 4: Between-run deltas:
run-20250817-012528 minus run-20250817-053841.

Task	Δ Acc.	Δ ROC-AUC (macro)	Δ ROC-AUC (micro)
type	+0.0027	+0.0001	-0.0001
proj_orig	+0.0044	+0.0004	+0.0004
author	-0.0027	0.0000	0.0000

gap between *type* and *project/author* aligns with class-cardinality and imbalance (Section 3), reinforcing the use of macro-averaged metrics. The minimal impact of the optimizer suggests that early stopping dominates optimization dynamics under the present data scale and regularization. Consistent, modest gains attributable to pretraining are observed for *type* and *project*, whereas *author* remains more variable, likely due to residual stylistic noise after anonymization; macro-F1 and weighted-F1 are therefore more informative for this objective and should be prioritized in future analyses.

We report per-task validation metrics for successful runs, aggregating by parameter configuration rather than run identifiers. Configurations are defined by the tuple (*backbone*, *initialization*, *optimizer*, *learning rate*, *weight decay*). Beyond Accuracy and ROC-AUC (macro/micro), we include macro-F1 and weighted-F1 to account for class imbalance.

Table 5 summarizes mean $\pm$ std across successful runs that share the same configuration. We observe that ImageNet pretraining consistently improves *type* and *project* in macro-F1 and AUC, while *author* remains more variable, consistent with residual stylistic noise even after anonymization. Optimizer choice (Adam vs. SGD with momentum) has a smaller effect than initialization and backbone depth; differences appear mostly as small variations in convergence stability (std across runs). Table 6 highlights, for each task, the best macro-F1 configuration; these settings are used for subsequent qualitative analyses (Section 5).

Overall, macro-F1 tracks Accuracy for *type*, but diverges for *project* and *author*, reinforcing the need for macro-averaged metrics under long-tailed distributions.

Table 5: Validation metrics over successful runs grouped by parameter configuration (mean $\pm$ std).

Task	Configuration	#Runs	Acc. σ	F1 (macro)	F1 (weigh.)	AUC (macro)	AUC (micro)
author	- pretrained -@- wd=-	4	0.8662 ± 0.0050	0.8432 ± 0.0055	0.8660 ± 0.0049	0.9951	0.9965
author	- scratch -@- wd=-	4	0.8646	0.8421	0.8655	0.9947	0.9963
proj_orig	- pretrained -@- wd=-	4	0.8580 ± 0.0027	0.8346 ± 0.0025	0.8571 ± 0.0017	0.9955	0.9969
proj_orig	- scratch -@- wd=-	4	0.8513	0.8222	0.8521	0.9950	0.9964
type	- pretrained -@- wd=-	4	0.9298 ± 0.0040	0.8669 ± 0.0031	0.9294 ± 0.0033	0.9977	0.9989
type	- scratch -@- wd=-	4	0.9236	0.8634	0.9238	0.9974	0.9989

Table 6: Best configurations per task (by macro-F1) among successful runs.

Task	Configuration	F1 (macro)	Acc.	AUC (macro)	AUC (micro)
author	- pretrained -@- wd=-	0.8432 $\pm$ 0.0055	0.8662	0.9951	0.9965
proj_orig	- pretrained -@- wd=-	0.8346 $\pm$ 0.0025	0.8580	0.9955	0.9969
type	- pretrained -@- wd=-	0.8669 $\pm$ 0.0031	0.9298	0.9977	0.9989

5 Explainability (XAI)

We compute attribution maps with Integrated Gradients (IG) using Captum. For each task, we sample up to N validation images per class (configured via `xai.per_class_samples`) and attribute with respect to the model’s predicted label. We overlay normalized IG attributions onto the input minimaps and include these panels in the run report. XAI is used solely for qualitative interpretation; it does not influence training or model selection.

Qualitative findings. Figure 3 illustrates qualitative attribution maps obtained with Integrated Gradients for the three classification objectives: (A) *author*, (B) *project*, and (C) *type*. (1) In *type*, IG concentrates on structural bands (indentation fields, delimiters), mirroring the strong quantitative separability; (2) in *project*, salient regions align with repository-specific conventions (headers/boilerplate), consistent with moderate accuracies and high ROC-AUC; (3) in *author*, attributions are more diffuse, reflecting stylistic factors that persist despite anonymization. Cross-run consistency in the saliency layout supports that the models rely on stable visual cues rather than dataset artifacts. These insights inform the design of imbalance-aware objectives and regularizers that penalize over-reliance on boilerplate while preserving structural signals.

Implications. The XAI evidence explains the relative task difficulty and suggests concrete avenues: (i) leverage pretraining for *type/project* where struc-

tural cues dominate; (ii) incorporate mixup/cutmix or focal loss to mitigate head dominance; (iii) add saliency-based constraints to reduce dependence on non-semantic artifacts (e.g., template headers) that may impair cross-repository generalization.



Fig. 3: Integrated Gradients (IG) attribution maps for the three classification objectives: (a) *type*, (b) *project*, and (c) *author*. IG uses a black baseline and 50 steps with input normalization matched to training; attributions are rescaled to $[0, 1]$ and converted to a negative imagem for better visualization.

Integrated Gradients (IG) attributions overlaid on minimaps. IG uses a white baseline and 50 steps with input normalization matched to training. Attributions are rescaled to $[1, 0]$ for visualization. For the *type* objective, concentrated bands often indicate structural elements characteristic of a file role; for *project*, recurring boilerplate regions highlight repository-specific conventions; for *author*, more diffuse patterns reflect stylistic regularities rather than isolated tokens.

6 Concluding remarks

This paper presented a large-scale study of *source code minimaps* spanning 685,906 images from 100 open-source projects and 10 programming languages, accompanied by a file-level catalog that enables reproducible joins across three objectives (*type*, *project*, *author*). A unified training protocol (i.e. fixed 128×128 inputs, end-to-end fine-tuning, early stopping, and macro/micro ROC-AUC reporting) supported systematic comparisons across backbones. Among the evaluated models, EfficientNet-B0 with ImageNet pretraining achieved the highest validation accuracies, with representative results of 0.9333 (*type*), 0.8604 (*project*), and 0.8705 (*author*), and ROC-AUC (macro/micro) consistently near or above 0.995. The relative difficulty across objectives aligns with corpus statistics: *type* benefits from strong structural cues, while *project* and *author* are moderated by cross-repository conventions and stylistic variability. Attribution analyses using Integrated Gradients yielded coherent, task-specific saliency patterns: structural bands and indentation fields for *type*; boilerplate and header regions for *project*; and more diffuse stylistic regions for *author*. Together, these findings indicate that minimaps preserve discriminative signals at multiple scales and that modern CNNs can leverage them effectively under severe class imbalance.

Several limitations remain. First, the *type* objective exhibits extreme skew, which, despite macro-averaged evaluation, can mask minority-class behavior. Second, author labels may conflate human and organizational/bot accounts, potentially introducing confounds at the style level. Third, fixed-resolution inputs standardize learning but may underutilize information present in especially long files. Finally, the present analysis focused on convolutional backbones; broader coverage (e.g., lightweight vision transformers) and cross-architecture ensembles were not explored here.

Beyond empirical results, the approach lends itself to everyday software-engineering workflows. Periodic, repository-wide snapshots of minimaps can help teams maintain architectural conformance by revealing small but systematic shifts in configuration templates, import patterns, or build scripts—signals of drift that are often missed when scanning raw diffs. The same visual regularities that characterize each repository can also guide continuous integration and code review: when a pull request introduces files whose minimaps diverge markedly from established repository signatures, reviewers obtain an immediate cue to prioritize scrutiny or to request justification for the deviation. Finally, aggregated author- and style-related regularities may assist incident triage by grouping suspicious changes during forensics; however, such signals are imperfect proxies and should be handled with safeguards and clear ethical guidance to avoid misattribution.

These scenarios align directly with the three supervised objectives. Repository-specific signatures captured by the *project* objective underpin both drift monitoring and CI triage; the *type* objective clarifies whether unusual combinations of file roles are being modified together; and the *author* objective—used cautiously—offers additional context about stylistic departure. In short, minimaps provide compact, reproducible cues that integrate naturally with existing pipelines while keeping human oversight central to decision making.

6.1 Future work

Building on these results, we outline concrete directions:

- **Imbalance-aware learning.** Evaluate class-balanced reweighting, focal loss, logit-adjustment, mixup/cutmix, and deferred reweighting schedules specifically for the long-tail regimes observed in *type*. Report per-class and macro-averaged calibration alongside ROC-AUC and F1 to quantify minority-class reliability.
- **Architectures and pretraining.** Extend the benchmark with vision transformers and hybrid CNN-Transformer models, including self-supervised pretraining (e.g., contrastive objectives) directly on minimaps to reduce reliance on natural-image priors.
- **Resolution and multi-scale inputs.** Compare fixed 128×128 to pyramidal or patch-based inputs, and investigate adaptive cropping strategies guided by XAI to emphasize structurally salient regions without inflating training cost.

- **Data curation and splits.** Refine label hygiene for *author* by separating organizational bots from human contributors; design time-aware and repository-disjoint splits to control leakage across *project/author*. Leverage the catalog to produce task-specific balanced subsets and to probe domain shift across ecosystems.
- **XAI-guided modeling.** Incorporate attribution-driven regularizers (e.g., penalties for over-reliance on boilerplate regions) and concept-based analyses (e.g., TCAV) to connect saliency with interpretable code concepts. Use counterfactual editing on minimaps to test causal sensitivity to formatting and header templates.
- **Broader tasks and metadata.** Explore multi-task learning across *type*, *project*, and *author*; integrate repository metadata (contributors, activity, size) as auxiliary signals; and extend evaluation to quality-related tasks (e.g., defect-proneness proxies, architectural stereotype detection).
- **Reproducibility and release.** Package preprocessing, catalog generation, and reporting pipelines as a public toolkit, facilitating community extensions beyond the 100-project corpus and enabling standardized comparisons.

An additional direction for future work is the integration of a preprocessing step that removes comments from the source code before generating minimaps. By filtering out natural language text embedded in comments, the resulting visual signatures will focus solely on software artifacts (structural and syntactic elements). This refinement is expected to reduce noise, avoid spurious attribution to stylistic comment patterns, and provide a cleaner basis for both classification and interpretability analyses.

In summary, the dataset, protocol, and analyses establish a reproducible reference for learning with visual abstractions of code. Scaling imbalance-aware training, diversifying architectures and pretraining, and closing the loop between XAI and model design are promising steps toward robust, interpretable software-visualization pipelines.

Acknowledgments. We thank the Brazilian research support agencies: Coordination for the Improvement of Higher Education Personnel - Brazil (CAPES) - Finance Code 001, Fundação Araucária for funding (Grant No. PRD2023361000043) and National Council for Scientific and Technological Development (CNPq) for their financial support.

Disclosure of Interests. The authors declare no conflicts of interest.

References

1. Adadi, A., Berrada, M.: Peeking inside the black-box: A survey on explainable artificial intelligence (xai). *IEEE Access* **6**, 52138–52160 (2018). <https://doi.org/10.1109/ACCESS.2018.2870052>
2. Aniche, M., Bavota, G., Treude, C., Gerosa, M.A., van Deursen, A.: Code smells for model-view-controller architectures. *Empirical Software Engineering* **23**(4), 2121–2157 (2018). <https://doi.org/10.1007/s10664-017-9540-2>

3. Anonymized Anonymized, A., Anonymized: Anonymized. In: Anonymized. Anonymized (Anonymized), anonymized
4. Costa, Y.M., Oliveira, L.S., Koerich, A.L., Gouyon, F.: Music genre recognition using spectrograms. In: International Conference on Systems, Signals and Image Processing (IWSSIP). pp. 1–4. IEEE (2011)
5. Costa, Y.M., Oliveira, L.S., Koerich, A.L., Gouyon, F., Martins, J.G.: Music genre classification using LBP textural features. *Signal Processing* **92**(11), 2723–2737 (2012)
6. Costa, Y.M., Oliveira, L., Koerich, A.L., Gouyon, F., Martins, J.G.: Music genre classification using lbp textural features. *Signal Processing* **92**(11), 2723–2737 (2012)
7. Costa, Y.M., Oliveira, L.S., Koerich, A.L., Gouyon, F.: Music genre recognition using spectrograms. In: 2011 18th International conference on systems, signals and image processing. pp. 1–4. IEEE (2011)
8. He, H., Garcia, E.A.: Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* **21**(9), 1263–1284 (2009). <https://doi.org/10.1109/TKDE.2008.239>
9. Kobayashi, K., Kamimura, M., Yano, K., Kato, K., Matsuo, A.: Sarf map: Visualizing software architecture from feature and layer viewpoints. In: IEEE International Conference on Program Comprehension (ICPC). pp. 43–52. IEEE (2013)
10. Novais, R.L., Nunes, C., Garcia, A., Mendonça, M.: Sourceminer evolution: A tool for supporting feature evolution comprehension. In: IEEE International Conference on Software Maintenance (ICSM). pp. 508–517. IEEE (2013)
11. Novais, R.L., Nunes, C., Garcia, A., Mendonça, M.: Sourceminer evolution: A tool for supporting feature evolution comprehension. In: 2013 IEEE International Conference on Software Maintenance (ICSM). pp. 508–517. IEEE (2013). <https://doi.org/10.1109/ICSM.2013.83>
12. Ortigosa-Hernández, J., Inza, I., Lozano, J.A.: Measuring the class-imbalance extent of multi-class problems. *Pattern Recognition Letters* **98**, 32–38 (2017)
13. Sundararajan, M., Taly, A., Yan, Q.: Axiomatic attribution for deep networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 70, pp. 3319–3328. PMLR (2017), <http://proceedings.mlr.press/v70/sundararajan17a.html>